Full Stack Developer Assignment: Task Management System with Authentication and Analytics

Project: Advanced Task Management and Productivity Dashboard

> Don't assume it to be a company project as its just to evaluate your skills in depth. Use of AI is allowed at some extent till you have knowledge of what you have done and how exactly the things are working. Any vague answer will lead to disqualification.

Objective

Build a full stack task management system with authentication, historical tracking, and analytics. The application must demonstrate structured backend architecture, secure JWT implementation, SQL database design, and a well-organized frontend.

This assignment is intended to evaluate system design thinking, clean code practices, database modeling, API design, and frontend architecture.

Technical Stack

Backend (Choose One)

- Django
- FastAPI
- Flask

Frontend (Choose One)

- React
- Next.js
- Vue

Database

- PostgreSQL (preferred)
- MySQL / SQLite

Styling

- Tailwind CSS
- Material UI
- shadcn/ui

- Or any modern UI framework

TypeScript or JavaScript is allowed.

Functional Requirements

1. Authentication and Authorization

Implement JWT-based authentication.

Required features:

- User registration
- User login
- Access token (short-lived)
- Refresh token (long-lived)
- Token refresh endpoint
- Password hashing (bcrypt or argon2)
- Protected routes on backend
- Protected routes on frontend

Explain your JWT flow in the README.

---

2. Database Design

Design a relational database schema with proper normalization and relationships.

Required entities:

User

- id
- name
- email
- password
- created_at

Task

- id
- user_id (foreign key)
- title

- description
- priority (Low / Medium / High)
- status (Pending / Completed / Archived)
- due_date
- created_at
- updated_at

TaskHistory

- id
- task_id (foreign key)
- action_type (created / updated / completed / deleted)
- previous_state (JSON or equivalent)
- timestamp

Feedback

- id
- user_id (foreign key)
- task_id (optional foreign key)
- comment
- rating (1–5)
- created_at

Provide:

- ER diagram
- Explanation of relationships
- Any indexing decisions

---

3. Task Management

Users must be able to:

- Create tasks
- Update tasks
- Delete tasks
- Mark tasks as completed
- Archive tasks
- Filter tasks by priority, status, or date

- Paginate task lists

Every modification to a task must create a corresponding TaskHistory record.

---

4. Analytics and Dashboard

Create backend endpoints that calculate and return analytics data.

Required analytics:

- Total tasks
- Completed tasks
- Completion percentage
- Tasks completed per day/week/month
- Most productive day
- Average completion time
- Priority distribution
- Productivity score (define and explain your formula)

Frontend must visualize this data using charts.

Backend Architecture

Follow a structured architecture with clear separation of concerns.

Expected structure (example):

```
backend/
 ├── app/
 │   ├── api/
 │   ├── models/
 │   ├── schemas/
 │   ├── services/
 │   ├── repositories/
 │   ├── core/
 │   ├── config/
 │   └── utils/
 ├── migrations/
 └── entrypoint

```

Requirements:

- Environment-based configuration
- Input validation
- Proper error handling

- Clean RESTful API design
- Use of migrations
- API documentation (Swagger/OpenAPI preferred)

Frontend Requirements

The frontend must include:

Pages:

- Login
- Signup
- Dashboard
- Task management view
- Analytics view

Must implement:

- Protected routes
- API service layer abstraction
- Token refresh handling
- Form validation
- Loading and error states
- Responsive layout

Charts may use:

- Recharts
- Chart.js
- ApexCharts
- Or equivalent

Documentation Requirements

The submission must include:

1. README file containing:
    - Setup instructions
    - Environment variables
    - Database migration steps

    - Explanation of JWT implementation
    - Analytics logic explanation
    - Productivity score calculation logic
2. ER Diagram (image or PDF)
3. API documentation (Swagger/OpenAPI or equivalent)

Optional Enhancements (Bonus)

- Docker and docker-compose setup
- Unit tests
- Redis caching for analytics
- Background jobs
- Role-based access control
- Rate limiting
- Deployment link
- CI/CD pipeline
- Dark/light theme support

Evaluation Criteria

Backend architecture and structure – 20%
JWT implementation and security – 15%
Database design and SQL queries – 15%
Analytics logic – 10%
Frontend architecture – 15%
UI/UX quality – 10%
Code quality and maintainability – 10%
Documentation clarity – 5%

Submission Guidelines

- Provide a GitHub repository link.
- Ensure the project runs locally.
- Include setup instructions.
- Include database migration scripts.
- Optional: provide seed data.

- Deadline: 2 days.

This assignment is designed to evaluate practical full stack development skills, system design ability, and production-level thinking.